

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 14 JANUARY 2020 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY
COMMENTED

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

SIGNATURE:

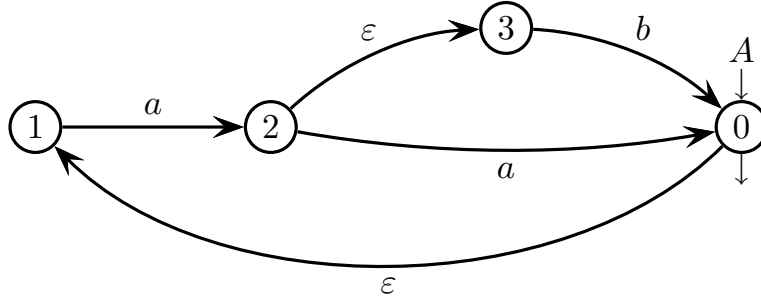
(or PERSON CODE)

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Take a two-letter alphabet $\Sigma = \{ a, b \}$. Consider the nondeterministic automaton A over Σ below (node 0 is both initial and final), which has spontaneous transitions (ε -transitions):



and consider also the regular expression R over Σ below:

$$R = (a a \mid a b \mid b a)^*$$

Answer the following questions:

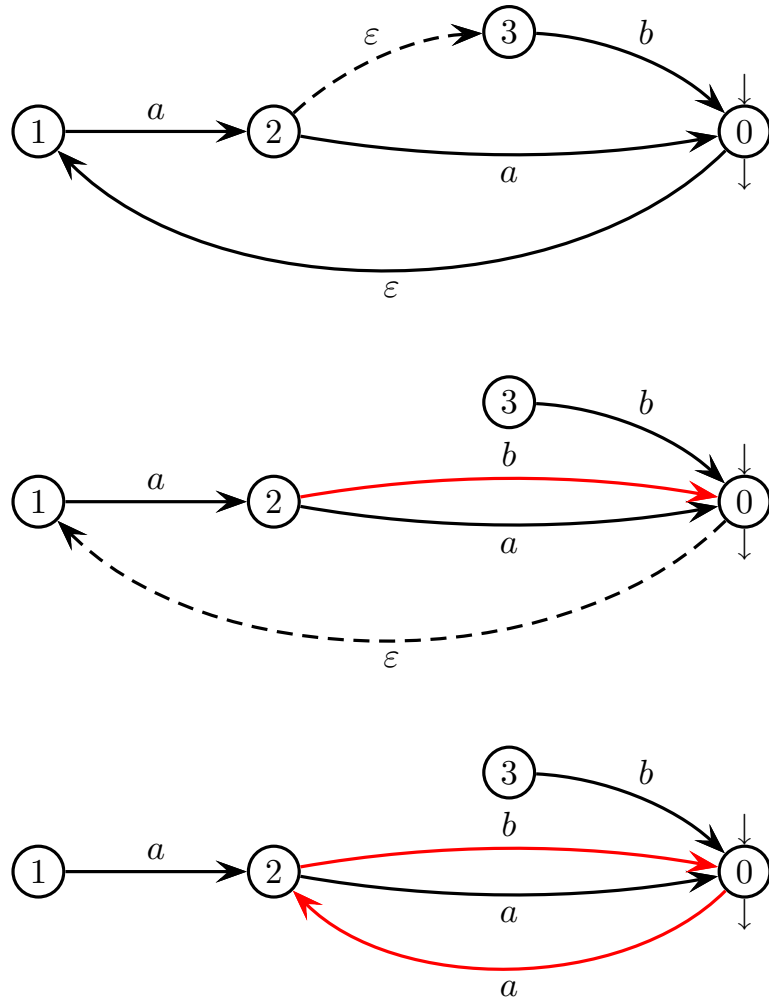
- (a) Find all the valid strings of language $L(A)$ that have a length ≤ 4 . Do the same for language $L(R)$, too.
- (b) By using the back-propagation method, eliminate the spontaneous transitions of automaton A and obtain an equivalent deterministic automaton A_1 without spontaneous transitions. Clean and minimize automaton A_1 , if necessary. Test automaton A_1 with the valid short strings found at point (a) for automaton A .
- (c) By using the Berry-Sethi method, obtain a deterministic automaton A_2 equivalent to the regular expression R . Minimize automaton A_2 , if necessary. Test automaton A_2 with the valid short strings found at point (a) for expression R .
- (d) Say if language $L(R)$ is local or not, and shortly justify your answer.
- (e) (optional) By using a systematic method of your choice, obtain a deterministic automaton A_3 for the difference language $L_D = L(R) - L(A)$, i.e., $L(R) \cap \overline{L(A)}$, which contains the strings generated by R that are not recognized by A . Then test automaton A_3 : by examining (only) the strings of length 4 found at point (a), find in alphabetical order the first two strings x and y such that $x \in L_D$ and $y \notin L_D$, and show that $x \in L(A_3)$ while $y \notin L(A_3)$.

Solution

(a) Both languages $L(A)$ and $L(R)$ are made of strings of even length: 0, 2, 4, Here are all such strings of length ≤ 4 , listed in alphabetical order:

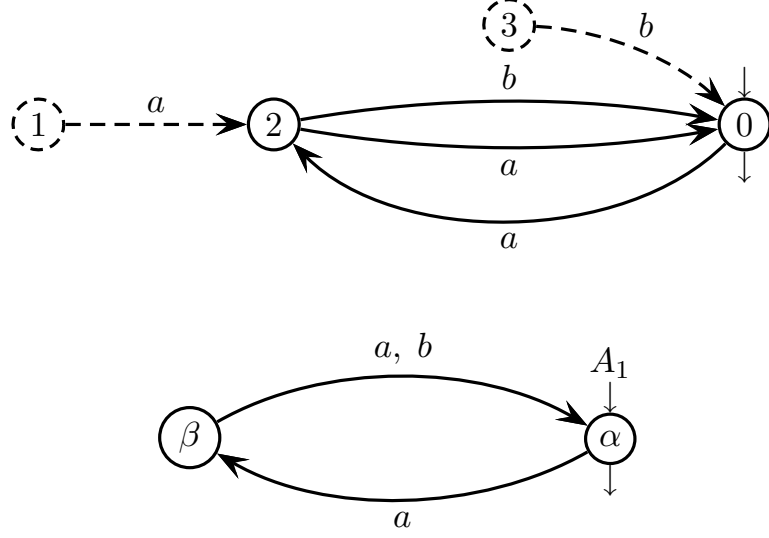
- short valid strings of $L(A)$: $\varepsilon, aa, ab, aaaa, aaab, abaa$ and $abab$
- short valid strings of $L(R)$: $\varepsilon, aa, ab, ba, aaaa, aaab, aaba, abaa, abab, abba, baaa, baab$ and $baba$

(b) Here is the construction of automaton A_1 , by eliminating the spontaneous transitions with the back-propagation method. First cut the ε -transitions:



The ε -arcs to be cut are dashed and the back-propagated arcs are in red.

Then, clean the useless states and transitions (dashed), and rename the states:

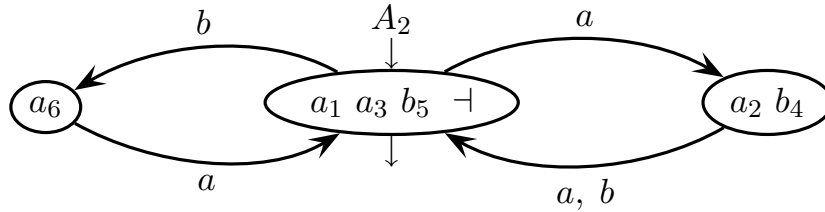


After cleaning, automaton A_1 is deterministic (by chance – in itself, cutting does not make it necessarily deterministic), has two states and is already minimal. In fact, states α and β are final and non-final, respectively, hence distinguishable. Testing the correctness of automaton A_1 is immediate: string ε is accepted as the initial state α is also final, strings aa and ab have the accepting paths $\alpha \xrightarrow{a} \beta \xrightarrow{a,b} \alpha$, and the strings of length four, i.e., $aaaa$, $aaab$, $abaa$ and $abab$, just repeat such cyclic paths, since all of them result from concatenating strings aa and ab . Therefore the test is passed successfully.

(c) Here is the construction of the deterministic automaton A_2 , by the *BS* algorithm:

$$R_{\#} = (a_1 a_2 \mid a_3 b_4 \mid b_5 a_6)^* \dashv$$

initials	$a_1 a_3 b_5 \dashv$
generators	followers
a_1	a_2
a_2	$a_1 a_3 b_5 \dashv$
a_3	b_4
b_4	$a_1 a_3 b_5 \dashv$
b_5	a_6
a_6	$a_1 a_3 b_5 \dashv$



The deterministic automaton A_2 is clean (by construction), has three states and is already minimal. In fact, the initial-final state is distinguishable from the other two non-final states, and state a_6 does not have the outgoing arc with label b , which state $a_2 b_4$ has instead, thus these two states are distinguishable as well, therefore automaton A_2 is the minimal one for the language.

(d) Language $L(R)$ is not local. Each one of the five arguments (i.A-B-C-D) and (ii) below reasonably suffices to disprove locality, by rising a contradiction:

i. Hypothetically assume that language $L(R)$ is local, thus automaton A_2 accepts a local language. From the regular expression R we can argue:

A. The digram bb is admissible, thus in a string three or more adjacent letters b are also possible. Yet automaton A_2 does not have three or more consecutive b -arcs, hence language $L(R)$ is not local.

B. Letter b is initial and final, and the digram bb is admissible. String bb is compliant with initials, finals and digrams, thus automaton A_2 should accept it. This does not happen, hence language $L(R)$ is not local.

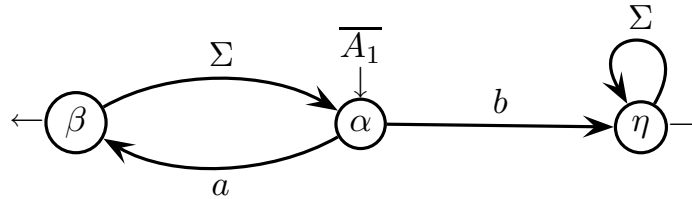
C. The local sets are $Ini = Fin = \Sigma$ and $Dig = \Sigma^2$, and the local language characterized by these sets is Σ^+ (any non-null string). Yet language $L(A_2)$ clearly differs from Σ^+ , hence language $L(R)$ is not local.

D. The local sets (i.C) allow strings of odd length, e.g., a , yet automaton A_2 does not accept them, hence language $L(R)$ is not local.

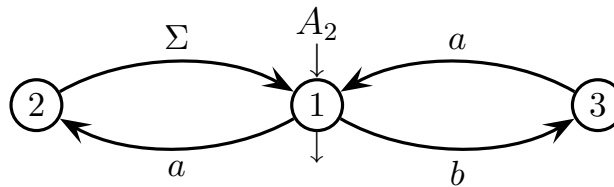
ii. The *minimal* automaton of a local language is necessarily local. Automaton A_2 is minimal, but it is not local. Thus language $L(R)$ is not local.

Other arguments may also exist, very different from the five above or simple variants thereof, e.g., (i.B) modified for string a , or b , or $aaa = a^3$, etc.

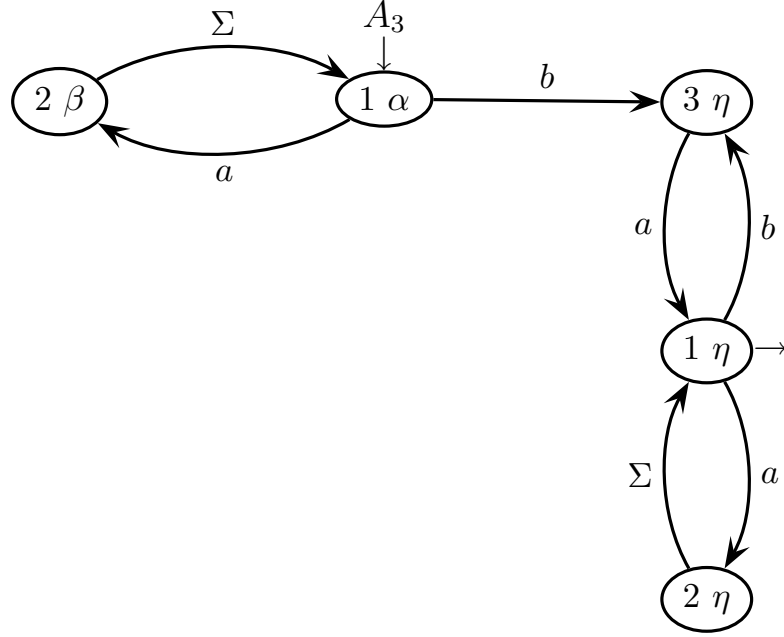
(e) Take the deterministic automaton A_1 , complete it with an error state η and complement it. Here is automaton $\overline{A_1}$, which is clean and even minimal:



Here is automaton A_2 , with node renaming and taking its symmetric image centered on the initial node, to expose its structural similarity to automaton $\overline{A_1}$:



Then intersect automata A_2 and $\overline{A_1}$. To do so, we could apply the standard product automaton construction $A_2 \times \overline{A_1}$, which would produce a state-transition graph with nine (i.e., 3×3) states (at most). Here however, we proceed more speedily: the left halves of both (deterministic) automata A_2 and $\overline{A_1}$ have identical nodes and interconnections, while state η is a trap and, once reached, it cannot be left any longer. Thus the product automaton A_3 reduces to:



Automaton A_3 has five states only. It is deterministic, as both factor automata are so. It is clean, though it might not be minimal (you may wish to check). Here is the test of automaton A_3 . From question (a), (i) alphabetically list the length-4 strings of language $L(R)$ and, similarly, (ii) those of language $L(A)$. Then in list (i) pick the first string x that is not in list (ii), and the first string y that is also in list (ii). Result: $x = aaba$ and $y = aaaa$. We soon see that $x \in L(A_3)$ and $y \notin L(A_3)$. Thus automaton A_3 passes this test successfully.

2 Free Grammars and Pushdown Automata 20%

1. Consider a two-letter alphabet $\Sigma = \{ a, b \}$. Then answer the following questions:

- (a) Write two *BNF* grammars G_1 and G_2 that generate the following two languages L_1 and L_2 , respectively:

$$L_1 = \{ w w^R \mid w \in \Sigma^* \}$$

$$L_2 = L(b^* a^* b^*)$$

- (b) Write a *BNF* nonambiguous grammar G_{12} that generates the intersection language $L_1 \cap L_2$. Argue that grammar G_{12} is not ambiguous.
- (c) Write a *BNF* grammar G_3 that generates the intersection language:

$$L_1 \cap L(b^* a^* b^* a^*)$$

Draw the syntax tree for a string (any string of your choice) of language $L(G_3)$.

- (d) (optional) Say if grammar G_3 is ambiguous, and suitably justify your answer.
-

Solution

- (a) Of course, the strings of language L_1 are the palindromes without a centre. Here is a quite standard grammar G_1 for L_1 (axiom S_1):

$$G_1 \begin{cases} S_1 \rightarrow a S_1 a \\ S_1 \rightarrow b S_1 b \\ S_1 \rightarrow \varepsilon \end{cases}$$

Language L_2 is regular by definition. Thus, here is an obvious right-linear grammar G_2 for L_2 (axiom S_2), which models the appropriate order of letters:

$$G_2 \begin{cases} S_2 \rightarrow b S_2 & \text{eliminated} \\ S_2 \rightarrow X & S_2 \rightarrow Y X Y \\ X \rightarrow a X & \text{unchanged} \\ X \rightarrow Y & X \rightarrow \varepsilon \\ Y \rightarrow b Y & \text{unchanged} \\ Y \rightarrow \varepsilon & \text{unchanged} \end{cases}$$

The variant aside is similar, though it is not entirely right-linear and is ambiguous (just think of how many trees each valid string of type b^+ may have).

- (b) Here is a grammar G_{12} for the intersection language $L_1 \cap L_2$ (axiom S_{12}):

$$G_{12} \begin{cases} S_{12} \rightarrow b S_{12} b & \text{unchanged} \\ S_{12} \rightarrow X & \text{unchanged} \\ X \rightarrow a X a & X \rightarrow a a X \\ X \rightarrow \varepsilon & \text{unchanged} \end{cases}$$

Intersecting the languages L_1 and L_2 requires identifying the palindromes of L_1 that are generated by the regular expression $b^* a^* b^*$ of L_2 . Thus, such strings must be of type $b^h a^k a^k b^h = b^h a^{2k} b^h = b^h (a a)^* b^h$, for any exponents $h, k \geq 0$. Grammar G_{12} is a restriction of grammar G_1 : it is structurally similar to G_1 , but it cannot generate a b -pair inside an a -pair, i.e., it excludes some derivations (or trees) of G_1 , and it produces all and only the strings of the required type. It is easily available in two versions, the variant aside with a right-linear rule.

Grammar G_1 is standard for palindromes and is known for being unambiguous. Since grammar G_{12} is a restriction of G_1 , with just a few trees removed, it is also unambiguous. The variant of G_{12} replaces a rule for nonterminal X with an equivalent one, of the right-linear type and also unambiguous.

- (c) Once more, we have to identify the palindromes that are generated by a regular expression, now $b^* a^* b^* a^*$. As a palindrome, the (regular) string will have to start and end either with a letter a or with a letter b . This means that it will have to be a palindrome either of type $b^* a^* b^*$ or of type $a^* b^* a^*$. Both cases are instances of the problem already solved in point (b). The only difference between them is that the roles of the letters a and b are swapped.

Therefore, a grammar G_3 for language L_3 is the following (axiom S_3):

$$G_3 \left\{ \begin{array}{l} S_3 \rightarrow X \mid U \quad \text{union rule} \\ \hline X \rightarrow b X b \mid Y \quad G_{12} \text{ instance (axiom } X) \\ Y \rightarrow a Y a \mid \varepsilon \\ \hline U \rightarrow a U a \mid V \quad G_{12} \text{ instance (axiom } U) - \text{swapped } a, b \\ V \rightarrow b V b \mid \varepsilon \end{array} \right.$$

Grammar G_3 is the union of two instances of grammar G_{12} (the variant with a right-linear rule would work as well) with swapped letters a and b .

The shortest string of language L_3 is the empty string ε . Thus, here are two derivations of grammar G_3 for ε , and their syntax trees (quite simple indeed):

$$\begin{array}{ccc} S_3 \Rightarrow X \Rightarrow Y \Rightarrow \varepsilon & \begin{array}{c} S_3 \\ | \\ X \\ | \\ Y \\ | \\ \varepsilon \end{array} & \begin{array}{c} S_3 \\ | \\ U \\ | \\ V \\ | \\ \varepsilon \end{array} \\ S_3 \Rightarrow U \Rightarrow V \Rightarrow \varepsilon & & \end{array}$$

- (d) Grammar G_3 is ambiguous because so is the empty string $\varepsilon \in L(G_3)$, since such a string has two syntax trees (see point (c)). In fact, the empty string is generated by both instances of grammar G_{12} . This is a case of ambiguity from union. On the other side, grammar G_3 is simple and immediate to obtain.

The reader may wish to extend the analysis and remove the ambiguity. All the strings of type $(aa)^+$ and $(bb)^+$, plus the empty string ε , are ambiguous of degree two like ε , essentially for the same reason as ε . In fact, both X and U are nullable and are able to generate all the strings that contain only one letter type, either a or b , while they differentiate if both letter types are present.

However, we can easily modify grammar G_3 and make it unambiguous. Here is an unambiguous version G_4 (axiom S_4):

$$G_4 \left\{ \begin{array}{l} S_4 \rightarrow X \mid U \mid \varepsilon \quad \text{union rule - directly nullable} \\ \hline X \rightarrow b X b \mid b Y b \quad X \text{ is unable to generate } (aa)^* \\ Y \rightarrow a Y a \mid \varepsilon \\ \hline U \rightarrow a U a \mid a V a \quad U \text{ is unable to generate } (bb)^* \\ V \rightarrow b V b \mid \varepsilon \end{array} \right.$$

Now the nonterminals X and U are non-nullable and are unable to generate the one-letter-type strings $(aa)^+$ and $(bb)^+$, respectively, while the axiom S_4 is still nullable as ε is valid. Of course, other grammar versions might exist.

2. We want to define a simplified language that describes the structure of certain flow graphs. The flow graph model to be considered is sketched below:

- A graph is a structure with a name, which is an identifier. A graph contains nodes and may contain arcs.
- A graph has exactly one input node and exactly one output node, both named and mandatory.
- A graph may have internal nodes. An internal node must have a name, which is an identifier.
- Two nodes may be connected by a (directed) arc, from the source node to the destination node. An arc may have a weight, which is a number.
- A graph may contain one or more subgraphs, which are graphs themselves, and so on at any containment depth.
- A subgraph, like a node, can be connected to nodes or subgraphs.
- The specification order of nodes and arcs is free, except that the input node precedes the output node. The subgraphs are listed after all the nodes and arcs.
- The lexical tokens, i.e., identifiers and numbers, are schematized by the terminals `id` and `num`, respectively, which do not need to be further expanded.
- For the language keywords and separators (and any other possibly unspecified details), refer to the sample description below as a guideline.

Here is a sample flow graph description:

```
graph MAIN is
  input is alpha;
  node is beta;
  arc from alpha to beta weighted 1;
  output is gamma;
  arc from beta to SUB;
  arc from SUB to gamma weighted 2;
  subgraph SUB is
    ...
  endsubgraph
  ...
endgraph
```

Answer the following questions:

- (a) Write a grammar, of type *EBNF* and not ambiguous, that generates the description language for the flow graph model sketched above.
- (b) Draw the (fragmentary) syntax tree of the sample graph description above.

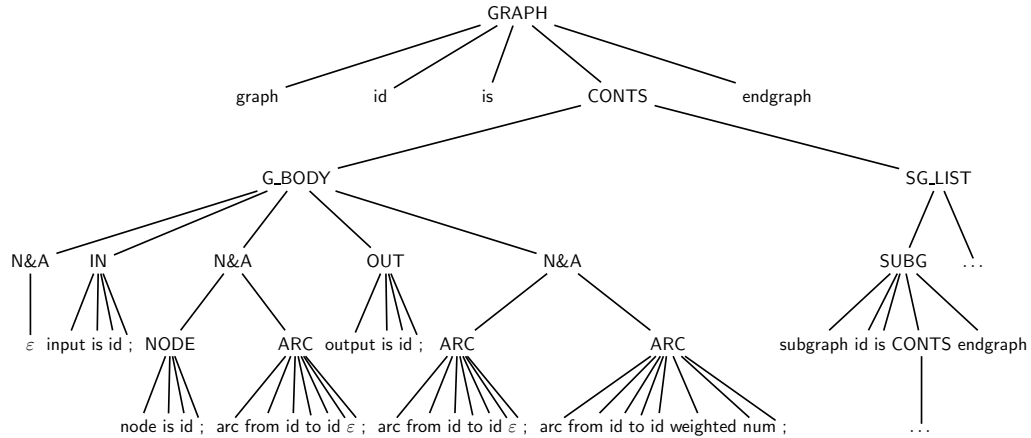
Solution

(a) Here is a possible formulation of the requested grammar (axiom GRAPH):

$$\left\{ \begin{array}{l}
 \langle \text{GRAPH} \rangle \rightarrow \text{graph id is } \langle \text{CONTS} \rangle \text{ endgraph} \\
 \langle \text{CONTS} \rangle \rightarrow \langle \text{G_BODY} \rangle \langle \text{SG_LIST} \rangle \\
 \langle \text{G_BODY} \rangle \rightarrow \langle \text{N\&A} \rangle \langle \text{IN} \rangle \langle \text{N\&A} \rangle \langle \text{OUT} \rangle \langle \text{N\&A} \rangle \\
 \langle \text{SG_LIST} \rangle \rightarrow \langle \text{SUBG} \rangle^* \\
 \langle \text{N\&A} \rangle \rightarrow (\langle \text{NODE} \rangle \mid \langle \text{ARC} \rangle)^* \\
 \langle \text{IN} \rangle \rightarrow \text{input is id ' ; ' } \\
 \langle \text{OUT} \rangle \rightarrow \text{output is id ' ; ' } \\
 \langle \text{SUBG} \rangle \rightarrow \text{subgraph id is } \langle \text{CONTS} \rangle \text{ endgraph} \\
 \langle \text{NODE} \rangle \rightarrow \text{node is id ' ; ' } \\
 \langle \text{ARC} \rangle \rightarrow \text{arc from id to id [weighted num] ' ; ' }
 \end{array} \right.$$

The grammar is *EBNF* and not ambiguous. In fact, there is no circular derivation, no regular expression in a rule right part is ambiguous, all the alternative nonterminals start with different keywords (*i*), e.g., $\langle \text{NODE} \rangle \mid \langle \text{ARC} \rangle$ with **node** | **arc**, and each nullable nonterminal has an initial keyword that distinguishes it from what follows (*ii*), e.g., $\langle \text{N\&A} \rangle$ (nullable) starts with **node** | **arc** and $\langle \text{IN} \rangle$ (following) starts with **input**, so that there may be neither union ambiguity (*i*) nor concatenation ambiguity (*ii*). Reasonably, the grammar is even of type *ELL* (the reader may wish to check), consequently not ambiguous.

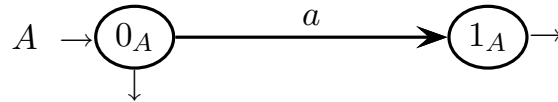
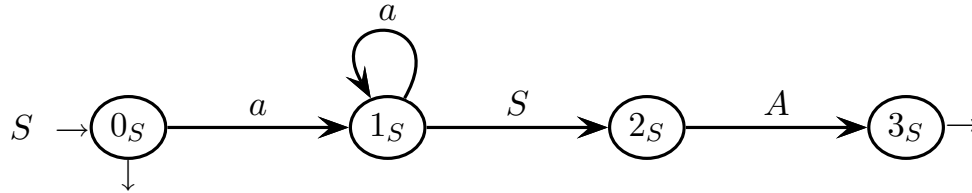
(b) Here is the fragmentary syntax tree of the sample graph description before:



The tree corresponds to the sample graph description. Notice that there is only one chain of recursive symbols, namely the tree branch $\text{CONTS} \Rightarrow \text{SG_LIST} \Rightarrow \text{SUBG} \Rightarrow \text{CONTS}$, which is self-embedding (autoinclusive) and not ambiguous. Furthermore, the tree reasonably demonstrates (or at least convinces of) the correctness of the grammar, since it exercises all the grammar rules.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, over the one-letter terminal alphabet $\Sigma = \{ a \}$ and the two-letter nonterminal alphabet $V = \{ A, S \}$ (axiom S):

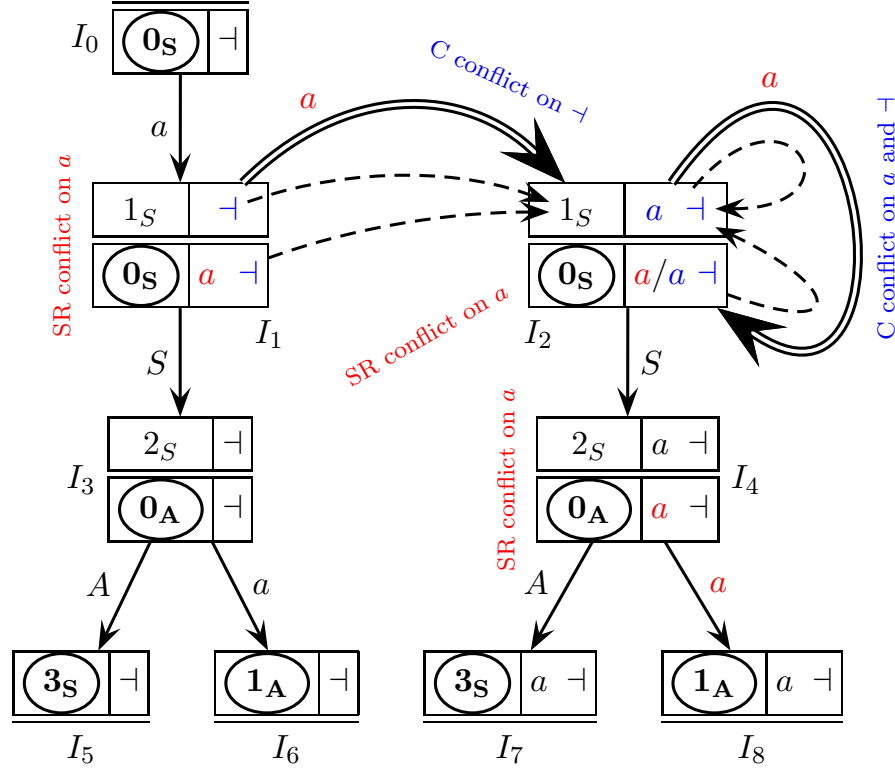


Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw the complete pilot automaton of grammar G . Determine whether grammar G is of type $ELR(1)$. Highlight all the conflicts that may be present in the pilot automaton, and indicate their type.
- (b) By finding all the guide sets on the call arcs and exit arrows, determine whether grammar G is of type $ELL(1)$. Justify your answer.
- (c) Draw the syntax tree (or all syntax trees if many exist) of the valid string aa .
- (d) (optional) Analyze the valid string aa by means of the Earley method, and highlight the item(s) showing that the string is accepted.

Solution

(a) Here is the complete pilot automaton of grammar G , with nine m-states:



The pilot has two multiple (more precisely double) transitions, namely $I_1 \xrightarrow{a} I_2$ and $I_2 \xrightarrow{a} I_2$, the latter one being a self-loop. The multiple transitions are double-lined to distinguish them from the single ones. Moreover, both such transitions are convergent, as their component sub-transitions from item to item (here drawn dashed) go to the same machine state 1_S in the m-state I_2 .

The pilot has a few conflicts (see the colors), here listed by their category:

shift-reduce in the m-state(s):

- I_1 and I_2 , both caused by the reduction items $\langle \overline{0_S}, \{a, \mid\} \rangle$ and by the shift arcs $I_1 \xrightarrow{a} I_2$ and $I_2 \xrightarrow{a} I_2$, respectively, on terminal a
- I_4 , caused by the reduction item $\langle \overline{0_A}, \{a, \mid\} \rangle$ and the shift arc $I_4 \xrightarrow{a} I_8$, still on terminal a

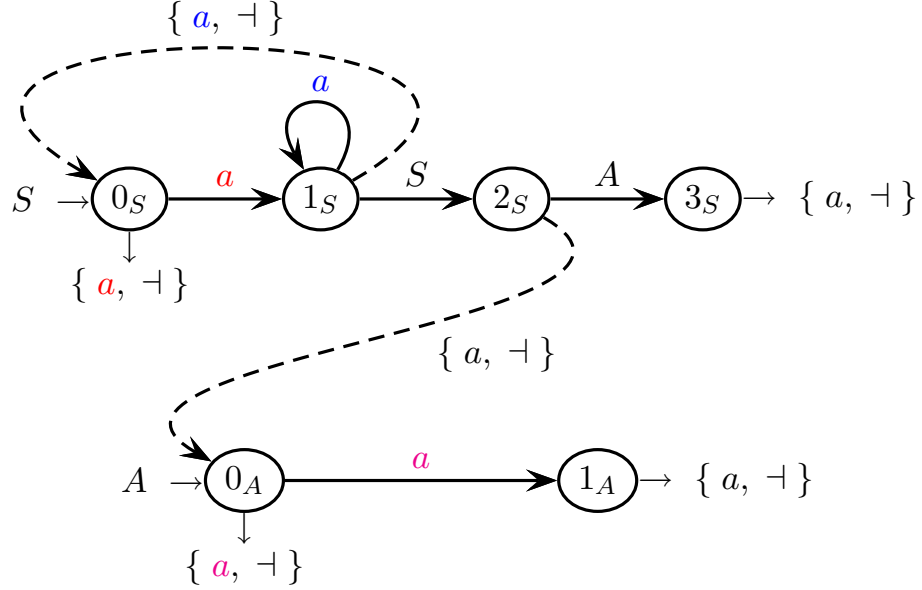
reduce-reduce none, as no m-state contains two or more reduction items

convergence on transition:

- $I_1 \xrightarrow{\mid} I_2$, caused by the terminator $\mid$, as both sub-transitions (dashed) share the look-ahead symbol $\mid$
- $I_2 \xrightarrow{a} I_2$, caused by terminal a and the terminator $\mid$, as both sub-transitions (dashed) share the look-ahead symbols a and $\mid$

Therefore, the pilot is not deterministic and grammar G is not of type $ELR(1)$. Notice that the pilot does not have the STP , despite every m-state has a base containing a single item. However, having all the m-states with a single-state base is a necessary condition for having the STP , but it is not a sufficient one.

- (b) Here is the *PCFG* of grammar G , with the guide sets on the call arcs and exit arrow (those on the terminal shift arcs are trivial and here are not shown):



Here is an explanation of how to compute the guide sets. First notice that both nonterminals S and A are nullable. These are the sets on the exit arrows (f_X represents any final state f of machine M_X , e.g., f_S stands for 0_S and 3_S):

$Gui(f_S \rightarrow)$ it contains the terminator $\perp$ by definition; it contains terminal a since the call site $1_S \xrightarrow{S} 2_S$ of S is immediately followed by A and $a \in Ini(A)$

$Gui(f_A \rightarrow)$ it includes the whole set $Gui(f_S \rightarrow)$ since the call site $2_S \xrightarrow{A} 3_S$ of A immediately goes to the final state 3 of M_S

and those on the call arcs:

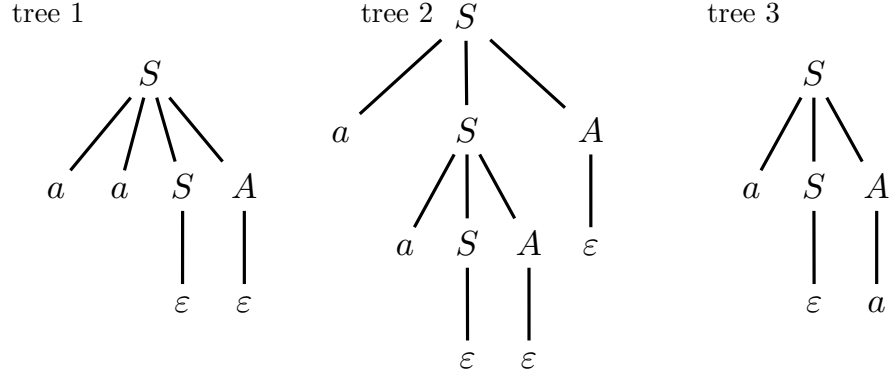
$Gui(1_S \xrightarrow{S} 0_S)$ it includes the whole set $Gui(f_S \rightarrow)$ since S itself is nullable and the call site $1_S \xrightarrow{S} 2_S$ of S is followed by the nullable path $2_S \xrightarrow{A} 3_S$ that goes to the final state 3 of M_S

$Gui(2_S \xrightarrow{A} 0_A)$ it includes the whole set $Gui(f_S \rightarrow)$ since A itself is nullable and the call site $2_S \xrightarrow{A} 3_S$ of A immediately goes to the final state 3 of M_S

It turns out that all the guide sets coincide with the terminal alphabet plus the terminator $\perp$. Remember that a symbol may belong to a guide set for more than one reason and that each such reason is sufficient to include the symbol. Thus the reason given for each set is sufficient, though others may exist.

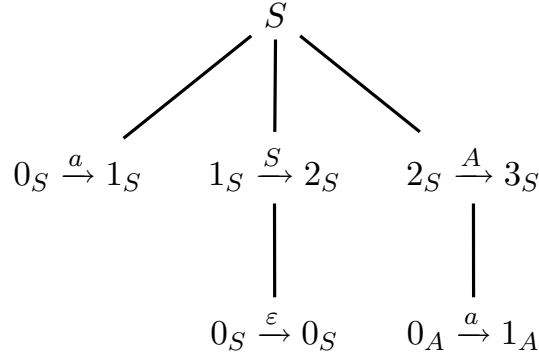
The *PCFG* of grammar G is not deterministic, because of the overlapping guide sets on the call arcs or on the exit arrows from states 0_S , 1_S and 0_A . All the conflicts are caused by terminal a (see colors). Thus grammar G is not of type $ELL(1)$. This was expected, as grammar G is not of type $ELR(1)$ either.

- (c) Here are all the syntax trees of the valid string aa generated by grammar G , which are three in total:



It turns out that grammar G is ambiguous. This is coherent with grammar G not being deterministic ELR , and consequently not ELL either. Longer valid strings of G have more and more trees, though still finitely many, thus grammar G has an unlimited ambiguity degree. Anyway, the ambiguity degree of G is not infinite, as there is not a valid string that has infinitely many syntax trees.

Notice however that the alphabet is one-letter, thus the context-free language $L(G)$ is surely regular. Consequently, there exists a different grammar G' , equivalent to grammar G but right-unilinear and deterministic. The reader may wish to find it (actually language $L(G)$ is simply Σ^* and a grammar G' is immediate). For completeness, here is the third tree (the smallest) with the machine paths:



Rewrite grammar G by *EBNF* rules and faithfully model the net, in this way:

$$G \left\{ \begin{array}{l} S \rightarrow a a^* S A \mid \varepsilon \\ A \rightarrow a \mid \varepsilon \end{array} \right.$$

Then, the (leftmost) derivation corresponding to the third tree is as follows:

$$S \xrightarrow{S \rightarrow a S A} a S A \xrightarrow{S \rightarrow \varepsilon} a \varepsilon A \xrightarrow{A \rightarrow a} a a$$

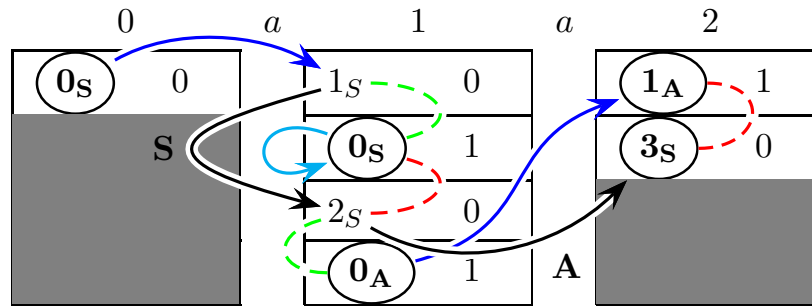
where the Kleene star in the *EBNF* rule of S is chosen to iterate zero times.

- (d) Here is the Earley (tabular) analysis of the valid string aa . Vector E has three states, which contain 1, 5 and 9 items, respectively:

$E(0)$	a	$E(1)$	a	$E(2)$
0_S 0		1_S 0		1_S 0
		0_S 1		1_S 1
		2_S 0		1_A 1
		0_A 1		0_S 2
		3_S 0		2_S 0
				0_A 2
				3_S 0
				2_S 1
				3_S 1

String aa is accepted because of the item $\langle 3_S, 0 \rangle$ (colored in red) with final state 3_S and pointer 0 in the vector last state $E(2)$. The prefix string a is accepted as well, in the vector state $E(1)$, because of the same item with final state 3_S and pointer 0 as before. Even the empty string ε is (trivially) accepted, in the vector initial state $E(0)$, in this case because of the item with final state 0_S .

The reader may wish to construct the three syntax trees of string aa , already shown for question (c). Here is how to construct the third tree (rightmost), the smallest one. Only the items that matter for such a tree are reported below:



A solid **blue**, **cyan** or **black** arrow represents a **non-null terminal** shift, a **null terminal** shift or a **nonterminal** shift, respectively. A dashed **green** or **red** line represents a **closure** or a **completion**, respectively. See also question (c).

4 Language Translation and Semantic Analysis 20%

1. Consider the binary alphabet $\Sigma = \{0, 1\}$ and a syntactic translation function τ on bit strings of even length ≥ 0 . Translation τ works as follows:

$$\tau(\varepsilon) = \varepsilon$$

$$\tau(b_1 b_2 b_3 b_4 \dots b_{2n-2} b_{2n-1} b_{2n}) = b_1 b_3 \dots b_{2n-1} b_{2n} b_{2n-2} \dots b_4 b_2$$

with $b_i \in \Sigma$, where $1 \leq i \leq 2n$ and $n \geq 1$. In words, the destination string consists of the odd-position symbols of the source string, followed by the even-position ones, the latter in reverse order. Here is an example:

$$\tau(011100) = 010011$$

Take the following source grammar G_s (axiom A) for the source language of τ :

$$G_s \left\{ \begin{array}{l} A \rightarrow 0 B \\ A \rightarrow 1 B \\ A \rightarrow \varepsilon \\ B \rightarrow 0 A \\ B \rightarrow 1 A \end{array} \right.$$

Answer the following questions (use the spaces / tables on the next page):

- (a) Without changing the source grammar G_s , write a syntactic translation scheme (or grammar) G_τ that computes translation τ . Test your scheme G_τ by drawing the destination (or translation) tree of the example above.
- (b) Consider the inverse translation τ^{-1} , which is computed by the scheme $G_{\tau^{-1}}$ with switched source and destination grammars. Answer these points:
 - i. Is translation τ^{-1} a function on strings? Shortly justify your answer.
 - ii. Compute the translation $\tau^{-1}(011100)$.
 - iii. Characterize how translation τ^{-1} works on a generic bit string of even length, in the same style as shown above for translation τ . Namely, complete this:

$$\tau^{-1}(b_1 b_2 \dots b_n b_{n+1} \dots b_{2n-1} b_{2n}) = \text{(write output at page bottom)}$$

- (c) (optional) Say if translations τ and τ^{-1} are deterministic of *ELL* type, for $k = 1$. Shortly justify your answers.

To be completed with the output string – see question (b) point (iii):

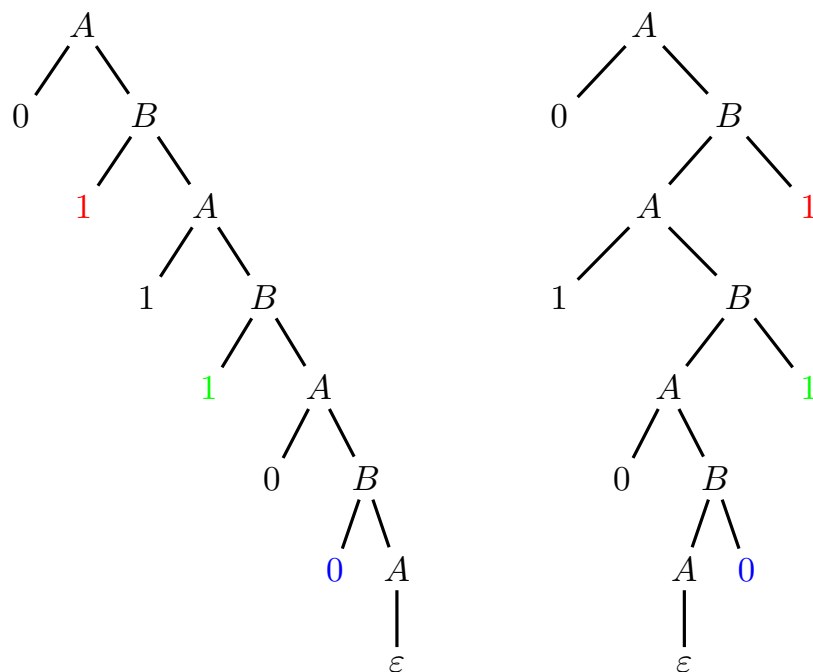
$$\tau^{-1}(b_1 b_2 \dots b_n b_{n+1} \dots b_{2n-1} b_{2n}) =$$

Solution

- (a) Mirror the source bits in the even positions, marked by nonterminal B . Here is a scheme G_τ (axiom A), with a destination / translation grammar G_d / G_τ :

$$G_s \left\{ \begin{array}{l} A \rightarrow 0 B \\ A \rightarrow 1 B \\ A \rightarrow \varepsilon \\ B \rightarrow 0 A \\ B \rightarrow 1 A \end{array} \right. \quad G_d \left\{ \begin{array}{l} A \rightarrow 0 B \\ A \rightarrow 1 B \\ A \rightarrow \varepsilon \\ B \rightarrow A 0 \\ B \rightarrow A 1 \end{array} \right. \quad \left| \quad G_\tau \left\{ \begin{array}{l} A \rightarrow \frac{0}{0} B \\ A \rightarrow \frac{1}{1} B \\ A \rightarrow \frac{\varepsilon}{\varepsilon} \\ B \rightarrow \frac{0}{\varepsilon} A \frac{\varepsilon}{0} \\ B \rightarrow \frac{1}{\varepsilon} A \frac{\varepsilon}{1} \end{array} \right.$$

Here are the source / dest. trees of the sample translation $\tau(0\mathbf{1}1\mathbf{1}00) = 0100\mathbf{1}\mathbf{1}$:



The colors highlight the correspondence between the source and destination bits. Both the source and the destination languages contain all the binary strings of even length, i.e., both are regular, but their translation relation is not rational.

(b) Here are the (quite simple) answers to the three points:

- i. The inverse translation τ^{-1} is a function on strings. This can be argued in (at least) two different ways:
 - Both τ and τ^{-1} are bit permutations, as they just change the bit positions. Permutations are known to be one-to-one functions (see below).
 - It suffices to notice that grammar G_d , which is source for τ^{-1} , is likewise unambiguous, thus τ^{-1} is a function. In detail, the argument runs as follows. Grammar G_d is source for τ^{-1} . Each bit in the tree of a string generated by G_d roots in that tree node one elementary subtree type, depending on the even or odd position of the bit in the string. Hence the string has only one tree. Thus grammar G_d is not ambiguous. Therefore, since τ^{-1} has a unambiguous source grammar and maps a bit to a bit (not to a set of two or more bits), it is a function.

The former argument examines the translation effect, a permutation, while the latter one examines the translation method, a syntactic scheme. The former is more general than the latter, as it would be applicable even if we did not know that the translation is defined by a syntactic scheme.

- ii. Translation τ^{-1} reverses the right half of the string and interleaves it in the even bit positions of the left half. The sample translation is as follows:

$$\tau^{-1}(011\mathbf{1}\mathbf{0}\mathbf{0}) = 0\mathbf{0}1\mathbf{0}1\mathbf{1}$$

- iii. Here is the characterization of translation τ^{-1} for a generic bit string:

$$\tau^{-1}(b_1 b_2 \dots b_n \mathbf{b_{n+1}} \dots \mathbf{b_{2n-1}} \mathbf{b_{2n}}) = b_1 \mathbf{b_{2n}} b_2 \mathbf{b_{2n-1}} \dots b_n \mathbf{b_{n+1}}$$

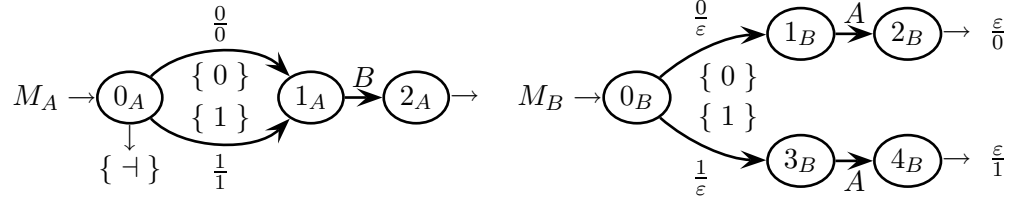
The right half is $b_{n+1} \dots b_{2n-1} b_{2n}$. It is reversed and interleaved bit by bit in the even bit positions of the left half.

Of course, the formula of point (iii) generalizes the example of point (ii).

- (c) Concerning the direct translation τ : the source grammar G_s is *LL* since the three alternative rules of A , as well as the two ones of B , have disjoint guide sets $\{0\}$, $\{1\}$ and $\{\neg\}$ (for $A \rightarrow \varepsilon$); thus τ is *LL* (and *a fortiori* it is *ELL*).

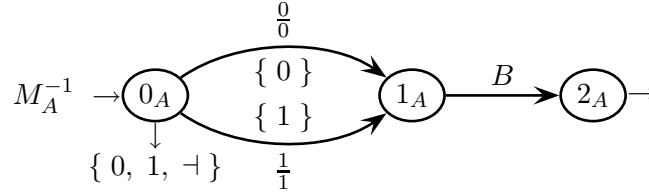
Concerning the inverse translation τ^{-1} : the destination grammar G_d , which is source grammar for τ^{-1} , is not *LL*, since the null rule $A \rightarrow \varepsilon$ has the guide set $\{0, 1, \neg\}$ overlapping with those of the other rules of A ; thus τ^{-1} is not *LL*. The status of τ^{-1} does not change in the *ELL* case.

For completeness, here is the exam of the *ELL* case. The translation net of τ is:



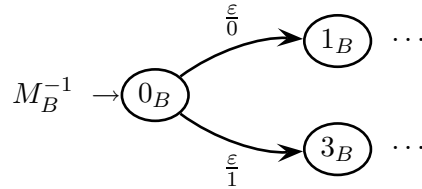
The call arcs are omitted and the guide sets are shown only on the bifurcation nodes 0_A and 0_B . Both machines M_A and M_B are modeled as *IO*-automata, with input over the alphabet of terminals and nonterminals, and output over the terminals, i.e., as rational transducers. Machine M_B also emits on exit. Both machines emit a single bit (or none) on each transition or on exit, not a set of two or more bits, i.e., both have one-valued output. Their underlying recognizers are deterministic, as none of them has a multiple transition from a state or has a spontaneous transition. Hence both machines, as pure rational transducers, are deterministic. Whence the translation net, as a *PCFG*, computes a function on strings. Furthermore, the guide sets on the nodes 0_A and 0_B are disjoint. Thus τ is *ELL*, and it is even *LL* as found before.

Take grammar G_d as source. The machine M_A^{-1} of the translation net of τ^{-1} is (just flip the fractions in the machine M_A of τ above – nothing changes):



Yet now the guide set on the exit arrow of state 0_A contains also 0 and 1, since in grammar G_d nonterminal A is followed by 0 and 1. Thus in M_A^{-1} the guide sets on state 0_A overlap and surely τ^{-1} is not *ELL* (nor *LL* as found before).

As M_A^{-1} fails to be *ELL*, designing a deterministic transducer M_B^{-1} is useless. The reader may wish to try anyway (flip the fractions in M_B), only to quickly find out that M_B^{-1} is not deterministic since it has ε -transitions (below):



Grammar G_d might still be *ELR*. This is unlikely anyway, since it is even doubtful whether the inverse translation τ^{-1} may be deterministic: it must interleave the right half of the string with the left half, long before reading the right half.

Of course, this could be done first by (nondeterministically) guessing and emitting the bits of the right half of the string when reading the left half, simultaneously storing each guessed bit onto the pushdown stack, and then by matching stack (in reverse order) and right half. Yet all this way is highly nondeterministic, and it is hard – if ever possible – to figure out how to determinize.

2. We intend to compute a semantic translation τ of an arithmetic expression from the infix form into a partial postfix form, by using an attribute grammar. Below it is reported the syntax of the (infix) expression to be translated (axiom S), which is right-associative, where terminal a represents any variable or constant:

$$\left\{ \begin{array}{l} 1: S \rightarrow E \\ 2: E \rightarrow T + E \\ 3: E \rightarrow T \\ 4: T \rightarrow (E) \\ 5: T \rightarrow a \end{array} \right.$$

Translation τ keeps all the parentheses as they are and works on the rest as follows:

- it leaves in *infix form* every (sub)expression (immediately nested in a parenthesis pair or at the outermost level) that has an *odd* number of terms (1, 3, ...)
- it puts in *postfix form* every (sub)expression (immediately nested in a parenthesis pair or at the outermost level) that has an *even* number of terms (2, 4, ...)

Here are two sample translations of increasing complexity:

$$\tau \left(\overbrace{a}^o +_1 \left(\overbrace{\overbrace{a}^o +_2 \overbrace{a}^e}^e \right) +_3 \overbrace{a}^o \right) = a +_1 \left(a a +_2 \right) +_3 a$$

$$\tau \left(a +_1 \left(a +_2 \left(a +_3 a \right) +_4 a \right) \right) = a \left(a +_2 \left(a a +_3 \right) +_4 a \right) +_1$$

The operator “+” is numbered to visualize how translation τ works. The position of each term is tagged odd or even (o or e) in each (sub)expression (top sample only) for the same purpose. Such annotations are not part of the attribute grammar.

The translated expression is represented by an attribute τ of type string in the root node of the syntax tree. On the next page, there is a list of grammar attributes (including τ) to use. Do not add or modify any attribute. There is also a decorated tree that illustrates how to use such attributes to translate a string (bottom sample).

Answer the following questions (use the tables / drawings / spaces on the next pages):

- Write the semantic functions for computing attribute *odd*.
- Write the semantic functions for computing attribute *totOdd*.
- Write the semantic functions for computing attribute τ .
- Write the translation of this expression (different from the two samples above):

$$(a + a + a) + a$$

and decorate its tree with all the appropriate attribute values in each node.

- (optional) Draw the dependency graph for each grammar rule.

<i>name</i>	<i>type</i>	<i>domain</i>	<i>symbol</i>	<i>meaning</i>
<i>odd</i>	right	boolean	<i>E</i>	true if the node occupies an odd position (first, third, etc) in the current sequence of nodes of type <i>E</i>
<i>totOdd</i>	left	boolean	<i>E</i>	true if the total number of nodes of type <i>E</i> in the current sequence is odd
τ	left	string	T, E, S	in the root, the translation of the entire tree; in the other nodes, suitable fragments of the overall translation

$$\tau\left(a+(a+(a+a)+a)\right)=a\left(a+(aa+)+a\right)+$$

$\tau = a(a + (aa+) + a) +$
 $totOdd = F$
 $\tau = \tau_1 \cdot \tau_2 \cdot +$
 $= a(a + (aa+) + a) +$

S
 E $odd = T$

$\tau = a$ T $+$ $totOdd = F$ $\tau = (a + (aa+) + a)$ E $odd = F$

a $\tau = (\cdot \tau_1 \cdot) = (a + (aa+) + a)$ T

$totOdd = T$ $\tau = \tau_1 \cdot + \cdot \tau_2$ $= a + (aa+) + a$ E $odd = T$

$\tau = a$ T $+$ $totOdd = T$ $\tau = \tau_1 \cdot + \cdot \tau_2$ $= (aa+) + a$ E $odd = F$

a $\tau = (\cdot \tau_1 \cdot) = (aa+)$ T $totOdd = T$ $\tau = a$ E $odd = T$

$totOdd = F$ $\tau = \tau_1 \cdot \tau_2 \cdot +$ $= aa+$ E $odd = T$ $\tau = a$ T

$\tau = a$ T $totOdd = F$ $\tau = a$ E $odd = F$ a

a $\tau = a$ T a

Solution

To help understanding, some elementary cases of τ are: $\tau(a) = a$, $\tau(a + b) = a b +$, $\tau(a + b + c) = a + b + c$ and $\tau(a + b + c + d) = a b c d + + +$, with named variables.

- (a) Here are the semantic functions for the right (inherited) attribute *odd*:

#	<i>syntax</i>	<i>semantics</i>
1:	$S_0 \rightarrow E_1$	$odd_1 := T$
2:	$E_0 \rightarrow T_1 + E_2$	$odd_2 := \underbrace{\neg odd_0}_{\text{toggled}}$
4:	$T_0 \rightarrow (E_1)$	$odd_1 := T$

The attribute is initialized to *true* at the first term of a (sub)expression (rules 1-4), and is toggled when moving on to the next term (rule 3), i.e., when moving down through the tree, until the last term of the (sub)expression is reached.

- (b) Here are the semantic functions for the left (synthesized) attribute *totOdd*:

#	<i>syntax</i>	<i>semantics</i>
2:	$E_0 \rightarrow T_1 + E_2$	$totOdd_0 := \underbrace{totOdd_2}_{\text{propagated}}$
3:	$E_0 \rightarrow T_1$	$totOdd_0 := \underbrace{odd_0}_{\text{init}}$

The attribute is initialized to the value assigned to attribute *odd* at the last term of a (sub)expression (rule 3), and is propagated unchanged (to translate uniformly) when moving back to the previous term (rule 2), i.e., when moving up through the tree, until the first term of the (sub)expression is reached.

- (c) Here are the semantic functions for the left (synthesized) attribute τ :

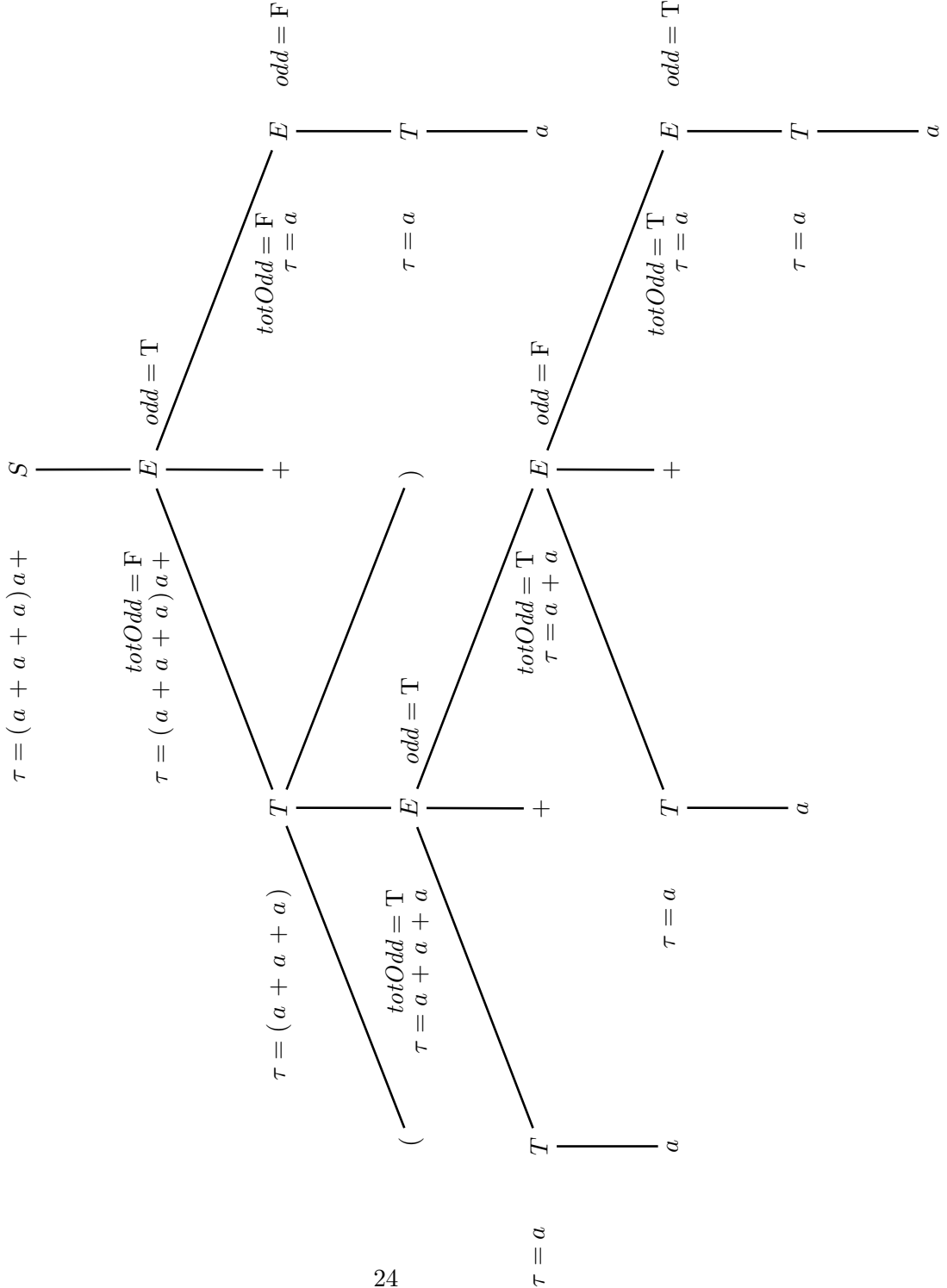
#	<i>syntax</i>	<i>semantics</i>
1:	$S_0 \rightarrow E_1$	$\tau_0 := \tau_1$
2:	$E_0 \rightarrow T_1 + E_2$	$\tau_0 := \text{if } \underbrace{totOdd_0}_{\text{select form}} \text{ then } \underbrace{\tau_1 \cdot + \cdot \tau_2}_{\text{infix form}} \text{ else } \underbrace{\tau_1 \cdot \tau_2 \cdot +}_{\text{postfix form}} \text{ end}$
3:	$E_0 \rightarrow T_1$	$\tau_0 := \tau_1$
4:	$T_0 \rightarrow (E_1)$	$\tau_0 := \underbrace{(\cdot \tau_1 \cdot)}_{\text{unchanged}}$
5:	$T_0 \rightarrow a$	$\tau_0 := a$

The attribute will stepwise accumulate the translation. It grows by concatenating a term and (the rest of) a subexpression when moving up through the tree, and by putting them in the infix or postfix form according to attribute *totOdd* (rule 2), while the parentheses do not change (rule 4). It is initialized to an elementary term (rule 5) and, as for the rest, it is propagated upwards (rules 1-3). In the root, it will have thus taken its final value, i.e., the complete translation.

- (d) The expression $(a + a + a) + a$ to translate consists of a 3-term subexpression $a + a + a$ and an outer 2-term expression $(\dots) + a$. The translation is:

$$\tau \left((a + a + a) + a \right) = (a + a + a) a +$$

The subexpression is left in infix form as it has three (odd) terms, i.e., $a + a + a$, while the expression is put in postfix form as it has two (even) terms, i.e., $(\dots) + a$ becomes $(\dots) a +$. The parentheses are untouched. Here is the decorated tree:



These elementary graphs are acyclic, and it is impossible to form a cycle by composing them on a tree. Thus the attribute grammar is (reasonably) correct.